

Algorithm Design for Brick Braker Game

Rehan Ahmad (Fall-23-Bscs-467) rehanamhad3812@gmail.com

Hammad Hanif (Fall-23-Bscs-469) hammadhanif0052@gmail.com

M.Fezan (Fall-23-Bscs-466) fezan1029@gmail.com

OVERALL SUMMARY:

→ Game Introduction:

Brick Breaker Game is a classic arcade-style game where players control a paddle to interact with a bouncing ball in a dynamic and colorful environment. The game features multiple levels, customizable settings, and engaging mechanics.

→ Objective:

The objective of **Brick Breaker Game** is to clear all the bricks on the screen by bouncing a ball off the paddle. Players must prevent the ball from falling off the screen while progressing through increasingly challenging levels.

→ Algorithmic foundation:

The **Brick Breaker Game** is built on foundational algorithms such as collision detection for ball and paddle interactions, brick breaking logic, and boundary constraints. The game also incorporates randomization for ball direction and structured data handling for level generation and scoring.

→ Difficulty system:

The game's difficulty system dynamically adjusts with each level, increasing the number of brick rows up to a maximum of 7 and introducing denser brick arrangements. The complexity of brick patterns add further challenges as players progress.

→ **Impact of algorithmic decisions on puzzle game design:**

Algorithmic decisions significantly influence puzzle game design by determining the game's mechanics, difficulty progression, and overall player experience. Efficient algorithms ensure smooth performance, enabling real-time interactions like collision detection and dynamic updates. Additionally, thoughtful design of randomization and state management enhances replayability and ensures fair yet challenging gameplay, creating an engaging and balanced puzzle-solving experience.

KEYWORDS:

1. Collision Detection
2. Game Design & Customization
3. Difficulty Progression
4. Leaderboard
5. Game Mechanics

INTRODUCTION:

→ **Brick Breaker** is a classic arcade game that originated from the widely popular game **Breakout**, developed by Atari in 1976. Inspired by the concept of Pong, Breakout introduced a new gameplay dynamic by replacing the second paddle with a wall of bricks for players to destroy. Over the years, this concept evolved, leading to modern adaptations like Brick Breaker, which incorporate additional features such as levels, power-ups, and improved graphics. Brick Breaker has remained a timeless favorite due to its simple yet addictive gameplay.

Rules of the Game

1. Players control a paddle that can move horizontally at the bottom of the screen.

2. A ball bounces around the screen and destroys bricks upon collision.
3. The objective is to break all the bricks on the screen to advance to the next level.
4. If the ball falls below the paddle, the player loses a life.
5. The game ends when the player loses all their lives or completes all levels.
6. Players earn points for each brick broken, with higher scores reflecting better performance.

→Problem Definition

The problem is to create a dynamic arcade game where a paddle, controlled by the player, bounces a ball to break bricks while simulating realistic physics and increasing difficulty.

Algorithm Definition

The algorithm manages ball movement, collision detection, game state updates, and level progression to ensure smooth and fair gameplay.

Goal of the Algorithm

To control game mechanics, detect interactions, and update the game state while providing an engaging experience.

Inputs and Outputs

- **Inputs:** Player controls, game environment (brick positions, ball direction).
- **Outputs:** Updated positions, score, level, and game visuals.

Assumptions and Constraints

- Paddle moves horizontally bricks break with one hit.
- Game entities stay within boundaries, and difficulty increases gradually.

→The expected performance of the algorithm is to ensure smooth, real-time gameplay by efficiently handling paddle and ball movements, collision detection, and state updates. It should:

1. **Accuracy:** Detect collisions precisely between the ball, paddle, walls, and bricks.
2. **Responsiveness:** Quickly respond to player inputs and update the game state without noticeable delays.
3. **Scalability:** Maintain performance across increasing levels of difficulty with more bricks and faster ball speeds.
4. **Consistency:** Provide predictable behavior to maintain fairness and enhance the player experience.
5. **Efficiency:** Minimize resource usage to ensure the game runs seamlessly on standard hardware.

→Test Cases and Scenarios

1. Paddle Movement

- **Input:** Press left or right arrow keys (or 'A'/'D' keys).
- **Expected Output:** Paddle moves horizontally within game boundaries.

2. Ball Movement and Collision Detection

- **Scenario 1:** Ball hits a brick.
 - **Input:** Ball collides with a brick.
 - **Expected Output:** Brick is destroyed, score increases, and ball bounces back.
- **Scenario 2:** Ball hits the paddle.
 - **Input:** Ball collides with the paddle.
 - **Expected Output:** Ball bounces in a direction relative to where it hit the paddle.
- **Scenario 3:** Ball hits a wall.

- **Input:** Ball collides with a wall.
- **Expected Output:** Ball bounces in the opposite direction.

3. Level Completion

- **Input:** All bricks are destroyed.
- **Expected Output:** Level increases, new bricks are generated, and the ball resets.

4. Losing a Life

- **Input:** Ball falls below the paddle.
- **Expected Output:** Life count decreases, ball resets, and game continues.

5. Game Over

- **Input:** Player loses all lives.
- **Expected Output:** Game ends, final score and level displayed, with an option to restart.

6. Leaderboard Update

- **Input:** Player completes a game session.
- **Expected Output:** Player score is saved to a file, and leaderboard displays scores in descending order.

7. Settings Adjustment

- **Input:** Player selects controls or background settings.
- **Expected Output:** Selected settings are applied and saved for gameplay.

8. Edge Cases

- **Scenario 1:** Ball is stuck in a corner.
 - **Expected Output:** Ball continues moving after collision resolution.

- **Scenario 2:** Simultaneous collisions (e.g., ball hits paddle and brick).
 - **Expected Output:** Game handles collisions without errors or inconsistencies.

ABOUT THE GAME:

→OBJECTIVE:

The objective of the Brick Breaker game is to break all the bricks on the screen by bouncing a ball off a paddle. Players need to control the paddle to keep the ball in play and hit all the bricks without letting the ball fall off the bottom of the screen. Successfully clearing all the bricks on a level allows the player to progress to the next level, with the ultimate goal of completing all ten levels of increasing difficulty.

→RULES:

1. Control the paddle using arrow keys or 'A' and 'D' keys to move left and right.
2. Keep the ball in play by bouncing it off the paddle.
3. Break bricks by hitting them with the ball.
4. Lose a life if the ball falls below the paddle.
5. Clear all bricks to advance to the next level.
6. Game ends when all ten levels are completed or all lives are lost.
7. Difficulty increases each level (more rows or more bricks).
8. Score points for each brick broken.
9. Use ESC key to return to the main menu at any time.

→GAME ELEMENTS:

1. Paddle: Player-controlled, moves horizontally at bottom

2. Ball: Bounces around, breaks bricks
3. Bricks: Arranged in rows, break when hit
4. Walls: Surround play area
5. Bottom wall: "Death zone" where lives are lost
6. Score display
7. Life counter
8. Level indicator

→**INITIAL STAGE:**

1. Paddle: Centered at bottom of screen
2. Ball: Above paddle, direction randomized
3. Bricks: 3-7 rows (based on level), randomly distributed
4. Lives: 5
5. Level: 1
6. Score: 0
7. Screen: 30 units wide, 27 units high
8. Walls: Present on sides and top
9. Player's nickname: Set before game starts

→**OBJECTIVE STATE:**

1. Clear all bricks from the current level
2. Advance through all 10 levels
3. Maintain at least one life throughout
4. Achieve the highest possible score
5. Complete level 10 to finish the game
6. Avoid letting the ball fall below the paddle

7. Break as many bricks as possible in each level
8. Progress through increasing difficulty levels
9. Aim for a high score to top the leaderboard

→**ACTIONS:**

1. Move paddle left (using left arrow or 'A' key)
2. Move paddle right (using right arrow or 'D' key)
3. Bounce the ball off the paddle
4. Aim the ball by hitting it with different parts of the paddle
5. Press ESC to return to main menu
6. Choose control scheme (arrow keys or 'A'/'D' keys)
7. Select background and asset colors
8. View leaderboard

→**MOVE VALIDATION:**

--**Paddle movement:**

1. Left movement is valid if paddle's left edge > 0
2. Right movement is valid if paddle's right edge $< \text{screen width} - 9$
3. Movement speed is constant (1 unit per move)

--**Ball movement:**

1. Valid if next position is within screen boundaries
2. Changes direction upon hitting walls, paddle, or bricks
3. Loses a life if touches bottom wall

-- **Brick interaction:**

1. Ball breaks a brick upon contact

2. Brick disappears and score increases

--Wall interaction:

1. Ball bounces off side and top walls
2. Bottom wall interaction results in life loss

-- Paddle-ball interaction:

1. Ball changes direction based on where it hits the paddle
2. Ball's direction randomized at the start of each level

-- Level progression:

1. Valid when all bricks are cleared
2. Game ends after completing level 10

-- Input validation:

1. Only specified keys (arrows or A/D) are valid for paddle movement
2. ESC key always valid for returning to main menu

→TRANSITION FUNCTION:

In Brick Breaker, the game state transitions occur based on player actions and game mechanics:

- Paddle Movement: The player moves the paddle left or right using arrow keys or 'A'/'D' keys, which updates the paddle's position on the screen.
- Ball Movement: The ball's position changes according to its direction and speed. If it collides with the paddle or bricks, its direction is adjusted, and the score increases.

- Life and Level Updates: If the ball falls below the paddle, the player's life decreases. Completing all levels of bricks advances to the next level, increasing difficulty.

→**GAME LOOP:**

1. Player Input: Capture player movements (left/right) to control the paddle.
2. Update State: Move the ball based on its direction and speed; check for collisions with the paddle and bricks.
3. Render Screen: Display the updated positions of the paddle, ball, and remaining bricks.
4. Check Game Status: Determine if all bricks are cleared or if the player has lost all lives.
5. Repeat: Continue the loop until the game objective is reached.

→**SCORING:**

- Brick Hit: Each time the ball hits a brick, 1 point is added to the total score.

→**END CONDITIONS:**

The game ends when either of the following conditions is met:

1. All Lives Lost: The player loses all their lives by failing to keep the ball in play.
2. All Bricks Cleared: The player successfully destroys all the bricks on the screen, completing the level.

→**USER INTERFERENCE:**

1. Customizable Interface: Players can change the background color and the colors of game components, including the ball, walls, paddle, and bricks, to personalize their gaming experience.
2. Game Display: The main screen shows the paddle, ball, and bricks, with a score counter at the top for easy visibility.

3. Controls: Players interact using keyboard arrow keys or 'A'/'D' keys to move the paddle left and right.

4. Game Over Screen: Upon game completion, a screen displays the final score and options to restart or exit.

→**Literature Review:**

The Brick Breaker genre, originating from the classic game Breakout released by Atari in 1976, revolutionized arcade gaming by introducing a paddle-and-ball mechanic that captivated players. Following its success, numerous clones and variations emerged, including Arkanoid in 1986, which added power-ups and enhanced graphics. The genre evolved further with titles like Ricochet and Shatter, incorporating 3D graphics and innovative gameplay elements.

In the 2000s, mobile gaming popularized the genre, leading to adaptations for smartphones and tablets, making it accessible to a wider audience. Modern iterations often feature customizable interfaces, diverse levels, and multiplayer options, reflecting advancements in technology and player preferences. The enduring appeal of Brick Breaker games lies in their simple yet addictive gameplay, blending skill and strategy, making them a staple in both casual and competitive gaming environments.

→**Problem Statement:**

The objective of the Brick Breaker game is to provide an engaging and interactive experience where players control a paddle to bounce a ball and destroy bricks on the screen. Players face challenges such as managing the ball's trajectory and dealing with varying brick configurations while aiming to achieve high scores. The game must ensure smooth gameplay, responsive controls, and customizable user interfaces to enhance player satisfaction. Additionally, implementing effective scoring and end conditions is crucial to maintain player motivation and provide a sense of accomplishment.

→**Algorithm Design Objectives:**

1. **Efficient Collision Detection:** Develop a robust algorithm to accurately detect collisions between the ball, paddle, and bricks, ensuring smooth gameplay and immediate feedback.
2. **Dynamic Ball Movement:** Implement a physics-based approach to calculate the ball's trajectory and speed, allowing for realistic bounces off surfaces and varied angles based on collision points.
3. **Scoring System Integration:** Design a scoring algorithm that updates the player's score in real-time as bricks are destroyed.
4. **User Customization:** Create a system that allows players to customize game components, such as colors for the background, ball, paddle, and bricks, while maintaining performance.
5. **Level Progression:** Design an algorithm that increases the difficulty of levels as players progress by introducing new brick patterns.
6. **Input Handling:** Develop a responsive input handling mechanism that captures player movements and actions accurately, ensuring the paddle responds intuitively to keyboard commands.
7. **End Condition Evaluation:** Implement checks to determine when the game ends, either through losing all lives or clearing all bricks, providing appropriate feedback to the player.

→**Proposed Solution:**

To fulfill the deliverables for the Brick Breaker game, the following solutions and strategies have been implemented:

1. **Game Mechanics Development:** Established core mechanics for paddle movement, ball physics, and collision detection, ensuring a smooth and engaging gameplay experience.
2. **User Customization Features:** Developed a settings menu that allows players to customize the background and component colors, enhancing user engagement and personalization.
3. **Scoring System Implementation:** Created a real-time scoring system that awards points for each brick hit, providing immediate feedback to players.

4. Level Design: Designed a level progression that introduces new challenges, such as varied brick layouts as players advance.

5. End Condition Logic: Implemented clear end conditions for the game, including checks for when all lives are lost or all bricks are cleared, ensuring a definitive conclusion to each play session.

6. User Interface Design: Developed a user-friendly interface that displays the score, lives, and customizable options, making it easy for players to navigate and interact with the game.

7. Testing and Debugging: Conducted thorough testing phases to identify and resolve bugs, ensuring a stable and enjoyable gaming experience before release.

8. Documentation: Created comprehensive documentation detailing the game mechanics, user interface, and customization options, facilitating future updates and enhancements.

These solutions collectively ensure that the deliverables are met, providing a polished and engaging Brick Breaker game experience.

→**TESTING AND ITREATION:**

Testing Plan

1. Unit Testing:

- Verify collision detection accuracy.
- Check scoring system calculations.

2. Integration Testing:

- Assess interactions between components (paddle, ball, bricks).
- Test user interface functionality.

Edge Cases to Consider

1. Ball Physics:

- Test behavior on corners and edges of bricks and paddle.

- Evaluate potential ball sticking scenarios.

2. Customization Limits:

- Ensure extreme color choices don't affect visibility.

3. Performance Under Load:

- Test performance with numerous bricks patterns.

4. End Condition Triggers:

- Verify correct triggering of game over conditions.

Iteration Process

1. Feedback Collection:

- Gather player feedback through surveys and interviews.

2. Refinement Based on Feedback:

- Prioritize changes based on common issues and suggestions.

3. Continuous Testing:

- Implement iterative cycles of testing and refinement.

4. Documentation of Changes:

- Keep records of changes and their rationale for future reference.

→OPTIMIZATIONS:

1. Collision Detection:

- Use spatial partitioning (quad-trees/grids).
- Implement bounding box checks.

2. Ball Physics:

- Simplify physics calculations.
- Ensure frame rate independence.

3. Memory Management:

- Minimize temporary object creation.

4. Dynamic Difficulty:

- Use adaptive algorithms based on performance.

→**CONCLUSION:**

Optimization and Enhancement Actions

1. Algorithm Refinement:

- Implement Spatial Partitioning: Optimize collision detection by integrating quad-trees or grid systems to reduce unnecessary checks.
- Adjust Physics Calculations: Simplify calculations for less critical interactions to improve performance.

2. Visual and Audio Enhancements:

- Add Background and Brick Images: Incorporate diverse visuals to enhance the game environment.
- Integrate Sound Effects: Implement sounds for collisions and brick destruction to improve player feedback.

3. Brick Durability and Scoring:

- Introduce Brick Endurance: Implement bricks with varying durability levels, awarding more points for tougher bricks.
- Enhance Scoring System: Adjust the scoring to reflect the new brick endurance mechanics.

4. Power-Up System:

- Develop Special Items: Create power-ups that players can collect, adding strategic elements to gameplay.

5. Visual Effects and Animations:

- Integrate Particle Effects: Add visual feedback for brick destruction to enhance player experience.

- Implement Dynamic Animations: Make movements more engaging with animations for the ball and paddle.

6. Level Design Adjustments:

- Create Themed Levels: Design levels with unique themes and layouts to keep gameplay fresh.

- Implement Progressive Difficulty: Gradually increase difficulty to maintain player engagement.

By following this structured approach, the algorithm will be effectively optimized and enhanced based on testing results and player feedback, leading to a more engaging and performant game experience.
